

Interrupts Revisited

1 Basic Idea

Definition

An **interrupt** is a hardware signal that causes the CPU to pause the currently running program and jump to an operating-system routine that handles an event, usually an I/O event.

- Interrupts let devices notify the CPU only when attention is needed.
- Without interrupts, the operating system would repeatedly poll devices.
- Interrupts are important for I/O because devices are much slower than CPU instructions.

Main Purpose

Interrupts allow asynchronous I/O: the CPU can run other work while devices operate, then react when a device signals completion.

2 Interrupt Hardware Structure

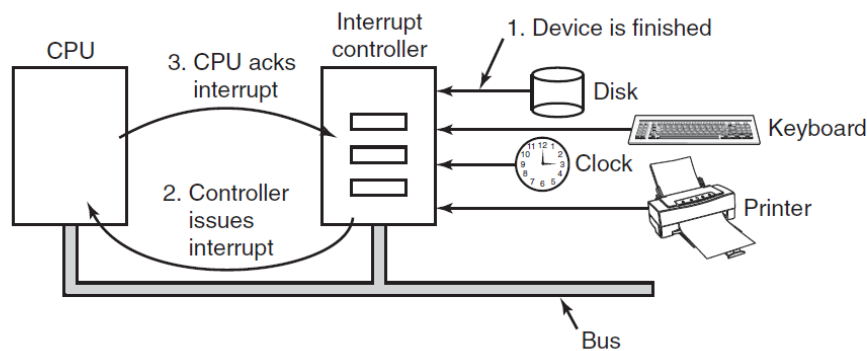


Figure 5-5. How an interrupt happens. The connections between the devices and the controller actually use interrupt lines on the bus rather than dedicated wires.

Component	Role
I/O device	Raises an interrupt request when it finishes work or needs attention.
Interrupt controller	Receives requests, resolves priority, and sends one interrupt signal to the CPU.
CPU	Stops normal execution, saves enough state, and starts the interrupt-service procedure.
Interrupt vector	Table used to find the correct interrupt-service procedure for the requesting device.

3 How an Interrupt Happens

Interrupt Path

The interrupt controller turns a device's request into a CPU-visible interrupt and identifies which service routine should run.

1. A device finishes its assigned work and asserts an interrupt request line.
2. The interrupt controller checks priority and whether another interrupt is active.
3. If the request can be handled, the controller accepts it.
4. The controller places a device-identifying number on the address lines.
5. The controller asserts the CPU interrupt signal.
6. The CPU uses the number as an index into the interrupt vector.
7. The CPU loads the new program counter from the vector entry and starts the interrupt-service procedure.

Interrupt Vector

The **interrupt vector** maps interrupt numbers to handler addresses. Traps and hardware interrupts often share this mechanism after the CPU has selected the vector entry.

4 Priority, Pending Requests, and Acknowledgement

Situation	Controller behavior
No interrupt is active	The controller can immediately signal the CPU.
Another interrupt is being handled	The new request waits until the controller is ready to issue it.
Two devices request together	The request on the higher-priority line is handled first.
Device is not yet serviced	The device continues asserting its interrupt signal.

Important Detail

When a lower-priority device is ignored temporarily, the request should not disappear. The device keeps asserting the interrupt until the CPU eventually services it.

Soon after the interrupt-service procedure starts, it writes a specific value to an I/O port of the interrupt controller. This is called **acknowledging the interrupt**. The acknowledgement tells the controller that it may issue another interrupt when appropriate.

5 State Saved by Hardware

Minimum Saved State

At minimum, the CPU must save the **program counter** (*PC*), because the *PC* tells the CPU where the interrupted program should continue.

Saved item	Why it matters
Program counter	Identifies where execution should resume.
Processor status word	Preserves condition codes, interrupt-enable bits, privilege state, and other CPU status.
General registers	Preserves the interrupted program's working values.
Internal state	May be needed on complex pipelined or superscalar CPUs.

6 Where to Save the State

Place	Advantage	Problem
Internal registers	Fast and avoids immediate memory writes.	A second interrupt may overwrite the saved state unless interrupts stay disabled.
Current stack	Simple because hardware can push state using the current stack pointer.	The stack may be a user stack, invalid, or near a page boundary.
Kernel stack	More likely to be valid, protected, and pinned in memory.	Switching to kernel state may require MMU work and can disturb cache and TLB contents.

Stack Hazard

If hardware saves interrupt state on an invalid user stack, the interrupt itself can cause a fatal error. If the save crosses a page boundary, it may cause a page fault while the CPU is already processing an interrupt.

7 Precise and Imprecise Interrupts

Precise Interrupt

A **precise interrupt** leaves the machine in a well-defined state, so the operating system can restart execution cleanly.

A precise interrupt has four required properties:

1. The *PC* is saved in a known place.
2. All instructions before the instruction pointed to by the *PC* have completed.
3. No instruction beyond the instruction pointed to by the *PC* has finished.

- The execution state of the instruction pointed to by the *PC* is known.

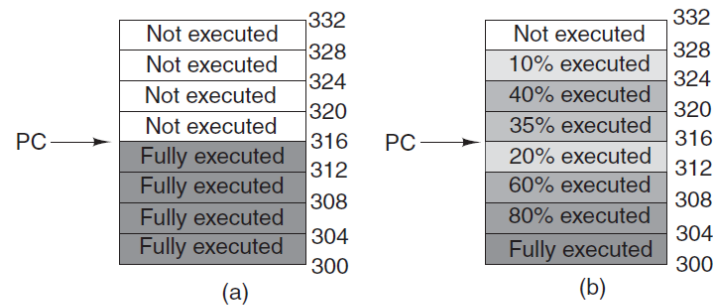


Figure 5-6. (a) A precise interrupt. (b) An imprecise interrupt.

Key Boundary

For a precise interrupt, the saved *PC* marks a clean boundary: instructions before it are complete, and later instructions have no visible completed effects.

8 Why Modern CPUs Make This Hard

- In a pipeline, different instructions occupy different stages at the same time.
- In a superscalar CPU, multiple instructions or micro-operations may execute together.
- Out-of-order execution can let a younger instruction nearly finish while an older one is still waiting.
- To provide precise interrupts, hardware may need retirement, rollback, logging, or shadow register logic.

Operating-System Difficulty

An **imprecise interrupt** does not give a clean restart boundary. The operating system may have to inspect saved internal CPU state to determine what completed and what still needs to be restarted.

9 Design Trade-Off

Type	Meaning and consequence
Precise interrupt	Clear restart boundary at the saved <i>PC</i> . Simpler for the OS, but hardware is more complex.
Imprecise interrupt	No clean boundary around the saved <i>PC</i> . Hardware may be simpler, but OS recovery is harder.

Performance Trade-Off

Precise interrupts move complexity into the CPU. Imprecise interrupts move complexity into the operating system.